Individual Project Guideline

Oct 13, 2026 · @thavorac chun

Each student builds, deploys and defends their own full-stack web application between week 3 and week 16, using Vue 3, NestJS and PostgreSQL.

Overview

You design and build one web application of your own choice, alone, over 14 weeks. It must solve a real problem for at least two kinds of users, store its data in a relational database, and run on a server that other people can reach.

The project is where you apply everything from the lectures. The practice sessions build one step of a reference app (a campus events booking app) each week; you then do the same step for your own app. Students who follow the practice sessions arrive at each milestone with a working base.

Layer	Technology	Taught in
Language	TypeScript (strict mode)	Weeks 1–3
Frontend	Vue 3, Vite, Vue Router, Pinia	Weeks 4–5
Backend	NestJS (REST API)	Week 6
Database	PostgreSQL with Prisma ORM	Week 7
Authentication	Password hashing, JWT, guards and roles	Week 8
Quality	ESLint, Jest/Vitest, Supertest	Weeks 3, 10
Deployment	Docker, docker compose, Nginx, GitHub Actions	Week 14

The project counts for **70%** of the course grade. The midterm exam after week 8 counts for the other 30%. There is no final written exam.

Requirements

Every item in the first table is required to pass. Pick at least one item from the second table.

Must have

Area	Requirement
Data model	At least 3 related entities in PostgreSQL, with at least one one-to-many relation; schema managed with Prisma migrations and a seed script

Area	Requirement
CRUD	Create, read, update and delete through the REST API wherever it makes sense for your domain
Users	Registration and login; passwords hashed (bcrypt or argon2), never stored in plain text
Authorisation	At least 2 roles (e.g. member and admin) and ownership checks: a user cannot edit or delete another user's data
Validation	On the client (form feedback) **and** on the server (DTOs with validation pipes); invalid requests get a 400 with a clear message
API design	Resource-based URLs, correct methods and status codes, consistent JSON error format, documented with Swagger
Frontend	Vue 3 + TypeScript SPA with routing, at least one Pinia store, route guards, and loading, empty and error states
Tests	Unit tests for core services and end-to-end tests for the main API endpoints
Security	Secrets in environment variables (never committed), CORS configured, no SQL injection or XSS
Deployment	Running on the school server behind Nginx, or, if the server is not available, starting with one `docker compose up`
Documentation	README with setup steps, architecture, environment variables and test accounts

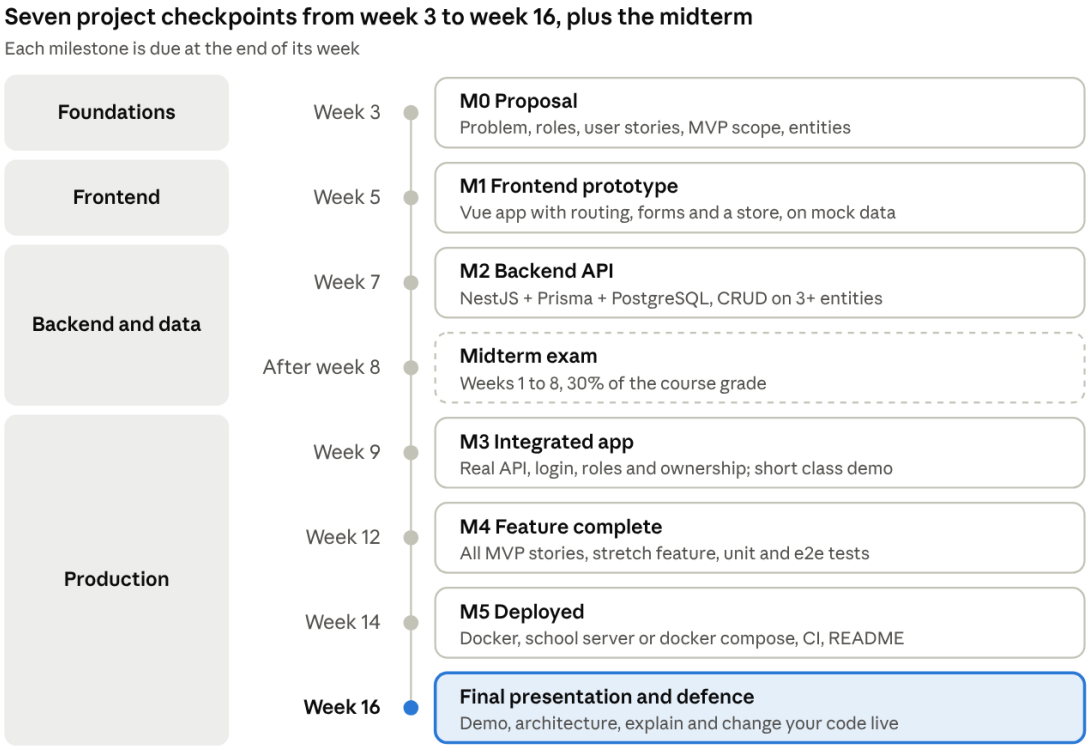
Stretch: choose at least one

Feature	Example
Search, filter and pagination	Find rooms by name and capacity, 10 per page
File upload	Profile picture or document attachment, with type and size limits
Real-time update	Live notification when a booking is approved (WebSockets)

Feature	Example
Scheduled task	Daily reminder or automatic cleanup of expired items

Milestones

The project runs in four phases that follow the lectures; each milestone matches what the practice sessions have just built.



project milestones · weeks 3 to 16

The practice session of the week a milestone is due is the last guided help for it; start each milestone as soon as its topic is taught.

Milestone checklists

Each milestone is due at the end of its week and submitted as a Git tag (`m0`, `m1`, …) on your repository. Each is graded on its checklist plus 2–3 short “explain your design” questions, answered in writing in `MILESTONES.md`.

M0 Proposal (week 3)

- ☐ `PROPOSAL.md` follows the template in Appendix A

- ☐ Problem stated in two sentences; target users and at least 2 roles named
- ☐ 6–10 user stories, with acceptance criteria for the 3 most important
- ☐ MVP scope split into must-have and nice-to-have
- ☐ Entity list with main attributes and relations; at least 3 entities
- ☐ Chosen stretch feature named
- Explain: Which story did you cut from the MVP, and why?

M1 Frontend prototype (week 5)

- ☐ Wireframes (photo or tool) of the main screens in `docs/`
- ☐ Vue 3 + TypeScript app created with Vite; strict mode on; ESLint passes
- ☐ At least 4 routes, including a detail page with a route parameter
- ☐ At least one form with client-side validation
- ☐ A Pinia store and a service layer calling a mock API (json-server or local JSON)
- ☐ Loading, empty and error states visible
- Explain: What state lives in the store and what stays inside a component? Why?

M2 Backend API (week 7)

- ☐ API contract (Appendix B) and ERD in `docs/`
- ☐ NestJS API with one module per resource (controller, service, DTOs)
- ☐ Prisma schema, migrations and a seed script; PostgreSQL runs in Docker
- ☐ CRUD endpoints for at least 3 entities, with server-side validation
- ☐ Correct status codes (200, 201, 204, 400, 404) and consistent error JSON
- Explain: Why did you choose this relation (1-N or N-N) between two of your entities?

M3 Integrated app (week 9)

- ☐ Frontend calls the real API; the mock is gone
- ☐ Registration, login, logout; passwords hashed
- ☐ Guards on the API for authentication, roles and ownership
- ☐ Route guards in Vue; the UI hides what a role cannot do
- ☐ Security checklist from week 9 run on your app; findings fixed or listed
- ☐ 3-minute demo in the practice session
- Explain: Where is the token stored in the browser, and what attack does that choice protect against or expose?

M4 Feature complete (week 12)

- ☐ Every MVP user story works end to end
- ☐ Stretch feature implemented
- ☐ Unit tests for core services; e2e tests for the main endpoints; all green
- ☐ Swagger documentation available
- Explain: Pick one test. What bug would it catch?

M5 Deployed (week 14)

- ☐ Dockerfiles for frontend and backend; `docker-compose.yml` for the whole stack
- ☐ Running on the school server with HTTPS, or starting locally with one `docker compose up`
- ☐ GitHub Actions workflow runs lint, tests and build on every push
- ☐ README complete (Appendix B); no secrets in the repository
- Explain: What happens, step by step, between `git push` and your change being live?

Final presentation and defence

In week 16 each student has about 20 minutes: a presentation in the lecture slot and a code defence in the practice slot. The code is frozen at the end of week 15 (tag `final`).

1. **Demo (5 min).** Show the main user stories live on the deployed app, logged in as each role.
2. **Architecture walkthrough (5 min).** One diagram of the system, the data model, and one design decision you would change if you started again.
3. **Code defence (10 min).** The lecturer picks a part of your code. You explain what it does and why, trace one request from the click to the database and back, then make a small change live (for example, add a field, a validation rule or a status check) and show it working.

Criterion	Excellent	Acceptable	Insufficient
Demo (20%)	All MVP stories work for every role, no errors	Main stories work, minor errors	Core features missing or crashing
Architecture (20%)	Clear diagram; justifies choices and names trade-offs	Describes the parts correctly	Cannot describe how the parts connect
Explaining code (30%)	Explains any chosen part precisely, including why	Explains with some hesitation or gaps	Cannot explain own code
Live change (30%)	Change done quickly, tested, nothing breaks	Change done with help or partly	Cannot make the change

A working application that the student cannot explain or change does not pass the defence.

Grading

The project is 70% of the course grade; the midterm exam is 30%.

Component	Course weight	How it is assessed
Midterm exam (after week 8)	30%	Written exam on weeks 1–8
Milestones M0–M5	20%	Checklist plus “explain your design” answers, about 3.3% each

Component	Course weight	How it is assessed
Final product	25%	Features, code quality, security, tests, deployment, Git history
Final presentation and defence	25%	Rubric in the previous section

Final product breakdown: features against the requirements 10%, code quality and structure 5%, security 4%, tests 3%, deployment and documentation 3%.

Rules and repository conventions

- **Individual work.** Your own idea, your own code. Discussing ideas with classmates is fine; sharing code is not.
- **Different domain.** Do not build a campus events or event booking app: that is the reference app in the practice sessions.
- **Explainable code.** You must be able to explain and modify every file in your repository. Code you cannot explain is treated as not yours.
- **Steady Git history.** Commit at least every week you work, with meaningful messages. A repository with one large commit before a deadline loses the Git history part of the final product grade.
- **No secrets in Git.** `.env` is in `.gitignore`; commit a `.env.example` with placeholder values instead. A leaked password or key must be changed, not just deleted.
- **Repository access.** Public on GitHub, or private with the lecturer added as a collaborator.

Suggested structure, one repository for the whole project:

```
my-project/
  web/            Vue 3 app (Vite)
  api/            NestJS app (with prisma/)
  docs/           wireframes, ERD, API contract, screenshots
  docker-compose.yml
  .github/workflows/ci.yml
  PROPOSAL.md
  MILESTONES.md
  README.md
```

Choosing an idea

A good project has 3–5 entities, 2–3 roles and clear rules about who can do what. Most failed projects were too big, not too hard.

Idea	Roles	Main entities	Stretch fit
Meeting room booking for a faculty	Staff, admin	User, Room, Booking	Real-time approval notice
Lost and found board for a campus	Student, moderator	User, Item, Claim, Category	Photo upload

Idea	Roles	Main entities	Stretch fit
Equipment lending for a lab	Student, lab assistant	User, Equipment, Loan	Scheduled overdue reminders
Recipe sharing with reviews	Cook, admin	User, Recipe, Ingredient, Review	Search and pagination
Small clinic appointment system	Patient, doctor, receptionist	User, Doctor, Appointment, Slot	Real-time queue
Club membership and dues tracker	Member, treasurer	User, Club, Membership, Payment	Search and pagination

Too small: a to-do list, a calculator, a personal blog with no roles. **Too big:** a social network with chat, video calls and payments.

You may change your idea until feedback on M0 in the week 4 practice session. After that, changes need the lecturer's approval.

Appendix A: PROPOSAL.md template

Copy this into `PROPOSAL.md` at the root of your repository and fill in every section.

```
# <Project name>

## Problem
Two sentences: who has the problem, and why it matters.

## Users and roles
- <Role 1>: what they do in the app
- <Role 2>: what they do in the app

## User stories
1. As a <role>, I want <goal> so that <benefit>.
   Accepted when: <condition>; <condition>; <condition>
2. ...
(6 to 10 stories; acceptance criteria for at least the 3 most important)

## MVP scope
Must have: story 1, 2, 3, ...
Nice to have: story 8, 9, ...

## Entities
| Entity | Main attributes | Relations |
|---|---|---|
| User | name, email, role | has many Bookings |
| ... | ... | ... |

## Stretch feature
One of: search + pagination, file upload, real-time, scheduled task.
What it does in this app:
```

```
## Repository
https://github.com/<you>/<project>
```

Appendix B: API contract and README templates

API contract (`docs/api.md`, due with M2). One row per endpoint; write it before the code.

```
| Method | Path | Who can call | Request body | Success | Errors |
|---|---|---|---|---|---|
| GET | /rooms?page=1 | anyone logged in | - | 200 Room[] | 401 |
| POST | /rooms | admin | { name, capacity } | 201 Room | 400, 401, 403 |
| PATCH | /bookings/:id | owner, admin | { start?, end? } | 200 Booking | 400, 401,
403, 404, 409 |
| DELETE | /bookings/:id | owner, admin | - | 204 | 401, 403, 404 |

Error format (all endpoints):
{ "statusCode": 400, "message": ["capacity must be a positive number"], "error": "Bad
Request" }
```

README (complete by M5).

```
# <Project name>
One sentence on what it does and for whom. Live URL (if deployed).

## Screenshots

## Architecture
Diagram or short description: web (Vue) -> api (NestJS) -> PostgreSQL.

## Run it locally
1. cp api/.env.example api/.env
2. docker compose up
3. Open http://localhost:5173

## Environment variables
| Name | Purpose | Example |

## Test accounts
| Role | Email | Password |

## API documentation
Swagger at /api/docs

## Tests
How to run unit and e2e tests.
```